Scholars Journal of Engineering and Technology
Abbreviated Key Title: Sch J Eng Tech
Journal homepage: https://saspublishers.com/journal/sjet/home

PLaND - Path to Least Non Determinism

Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo

Affiliations: Both authors are Independent Researchers, San Francisco, CA, USA. Corresponding author: Maddipatla Naga Venkata Sai Krishna; mnvsk97@gmail.com.

Abstract

Business processes and agentic systems contain decisions with different computational requirements. Some require interpretation, contextual judgment, novelty handling, or exception resolution; others are stable enough to execute deterministically. When these boundaries are not known in advance, a natural-language agent can provide an expressive starting point, but repeatedly routing already-stable work through a language model creates avoidable model calls and token consumption. We present Path to Least Non Determinism (PLaND), an evaluation-driven methodology for progressively reducing unnecessary model-mediated computation while preserving a predefined quality contract. PLaND begins with an entirely English SOP. Its evolver skill instructs a host reasoning agent to use development traces and scorer feedback to propose one bounded change, including explicit Python or Bash where appropriate; deterministic assessors then accept or reject the candidate under frozen comparison conditions. On the original untouched 1,000-case LEDGAR confirmatory test, natural-language and hybrid accuracy were 93.5% and 92.7%, respectively; the -0.8-point difference remained within the prespecified two-point non-inferiority margin, while model calls fell from 1,000 to 589 and tokens fell 40.02%. Three later optimized replications reproduced the -0.8-point difference and approximately 40.02% token reduction. A subsequent quality-first study, using fresh 500-case validation sets and stricter guardrails, rejected LEDGAR, CFPB, and SpamAssassin candidates; no test split was opened. These results support quality-gated deterministic substitution and fallback, not universal determinisation. They do not establish general autonomous pattern induction, continuous self-modifying workflows, or measured dollar savings.

Keywords: agentic workflows, deterministic compilation, language-model agents, business processes, workflow optimization, token efficiency, hybrid systems

Introduction

Business processes are not uniformly deterministic or uniformly agentic. A single process may include extraction, validation, classification, calculation, policy application, exception handling, contextual interpretation, and action selection. Different steps can require different computational representations. Some operations are deterministic from the outset. Some decisions remain dependent on semantic judgment. Other decisions may initially be handled by a language model because the stable structure of the problem is not yet fully known, but repeated execution can reveal patterns that are candidates for deterministic representation.

This distinction matters for agentic systems used in document processing, customer operations, compliance, underwriting, and fraud detection. In an unfamiliar fraud workflow, for example, the system may initially rely on model reasoning because relevant narratives and combinations of signals are not fully enumerated. Over time, recurrent high-confidence patterns may become sufficiently bounded to express as explicit rules, while novel or context-dependent cases continue to require model judgment. The systems problem is therefore not simply whether a workflow should be deterministic or non-deterministic. It is where non-deterministic reasoning remains necessary at a given stage of workflow maturity.

Language-model reasoning carries recurring expense. If a model is asked to rediscover the same stable decision boundary on every execution, the workflow repeatedly pays in model calls, tokens, and inference time for work that may no longer require open-ended reasoning. Under token- or call-based model pricing, reducing such work also provides a path to lower inference expense, although the experiments in this paper measure model calls and tokens rather than paid currency.

PLaND is motivated by this business and systems problem. Let a workflow be an ordered sequence W = (s1, s2, ..., sn). The representation of each step need not be fixed for the lifetime of the workflow. An English instruction can remain model-mediated when semantic judgment is required. A reference can organize substantial supporting procedure. A command can execute bounded and testable behavior directly. Moreover, a semantic step can remain hybrid: a deterministic command may resolve a validated subset of the input space and abstain on the remainder, which falls back to the model.

PLaND asks: **what is the least model-mediated form of the workflow that still satisfies its measured quality contract?**

The word *path* is intentional. PLaND does not assume that engineers know the final deterministic boundary before the workflow has been exercised. The initial generated SOP remains entirely in English. Development executions produce outputs, traces, latency, token use, cost proxies, and errors. The evolver skill instructs the host reasoning agent to cluster development failures and unnecessary expense and to propose one bounded change inside the workflow skill package. A candidate may introduce a reference or explicit Python/Bash command. Deterministic evaluation then decides whether the candidate is eligible for validation and eventual acceptance. If quality or guardrails fail, the prior accepted workflow is restored.

This architecture suggests a progression from open-ended model execution toward hybrid or increasingly structured execution as evidence permits. It does not imply that every workflow should eventually become deterministic. Novelty, ambiguity, contextual judgment, or distributional change may remain irreducibly model-mediated. The appropriate endpoint can therefore be an open-ended agent, a hybrid skill, or a mostly deterministic graph with model reasoning retained at selected boundaries.

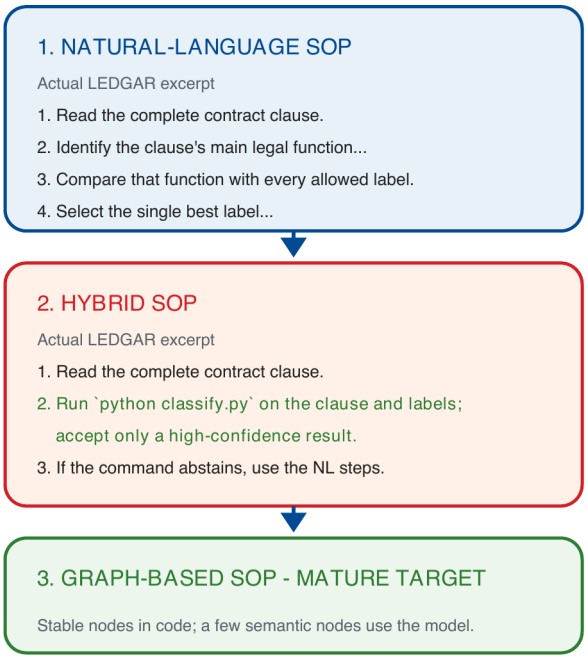

Figure 1. Actual LEDGAR SOP excerpts and the intended mature endpoint. The NL skill contains only English steps. The hybrid inserts `python classify.py` and returns to those English steps when the command abstains. The preferred mature endpoint, when validated, is a graph with only a few model-mediated nodes.

Reusable agent instructions are increasingly packaged as skills: the Agent Skills specification defines `SKILL.md` and supports references, scripts, and assets [1]; DeepAgents supports skill-based agent execution [2]; and LangGraph represents structured workflows as explicit graphs [3]. Related systems optimize language-model programs, prompts, workflows, or skills. DSPy compiles declarative model pipelines against metrics [4]; AutoFlow and AFlow generate and optimize workflows [5, 6]; Automated Design of Agentic Systems searches agent designs [7]; and SkillOpt, SkillRevise, SkillReducer, and ACES optimize or evaluate reusable agent capabilities [8-11].

PLaND addresses a narrower representation question: when can model-mediated work inside a workflow be replaced by deterministic execution under an explicit empirical contract? Its contribution is not a new general-purpose rule-mining algorithm. Rather, it combines an agent-guided candidate-evolution policy with deterministic evaluation and promotion boundaries. The current experiments test whether selected substitutions are safe and useful; they do not independently benchmark general autonomous discovery of rules from arbitrary histories.

The paper makes four contributions. First, it frames business and agentic workflows as compositions of decisions that can differ in their need for model reasoning. Second, it defines a trace-guided representation path from English instructions to references or commands, with the accepted English path retained as fallback for current-policy command candidates. Third, it freezes the experimental boundary and requires task-local quality and expense gates before promotion. Fourth, it reports successful, rejected, and nonviable cases across exploratory pilots, confirmatory studies, replications, and a stricter fresh quality-first validation, while separating demonstrated behavior from the broader longitudinal vision.

Material and Methods

Workflow and decision-region model

A business process may contain one or more workflows, and each workflow contains ordered steps W = (s1, ..., sn). PLaND treats the SOP step as its principal unit of analysis, but a step need not be wholly deterministic or wholly model-mediated.

For a semantic step si with input region Xi, consider a partition:

Xi = XD,i union XM,i,

where XD,i is a validated region that can be resolved by deterministic execution and XM,i is the residual region requiring model-mediated reasoning. The runtime behavior can be represented conceptually as:

fi(x) = gi(x), if the deterministic preconditions and guards hold;

fi(x) = Mi(x), otherwise.

Here gi is explicit code and Mi is the accepted model-mediated English path. A command can therefore abstain rather than forcing deterministic coverage. This is important for domains in which stable patterns coexist with novel or ambiguous cases.

PLaND supports two practically different forms of compilation. **Operation-level compilation** replaces stable preprocessing, transformation, validation, or tool work with deterministic code while leaving the semantic decision model-mediated. **Decision-region compilation** lets deterministic code itself resolve a validated subset of a semantic decision and sends the residual region to model reasoning. The document-classification schema-v2 pilot is primarily operation-level; the LEDGAR hybrid is primarily decision-region-level.

The broader longitudinal idea is that the validated deterministic region may change as new evidence becomes available. However, the current implementation does not mutate production logic continuously. Each PLaND experiment is bounded by a frozen evaluation contract. A later workflow version should therefore be treated as a new, separately validated evolution cycle rather than an unobserved online rewrite.

PLaND implementation

PLaND uses two Agent Skills. `generate-initial-version` creates a minimal DeepAgent containing exactly one workflow-named SOP skill. It derives task structure from requirements, datasource files, and evaluation schema, but the baseline SOP remains entirely English. Each step receives a stable identifier such as `[S01]` and an English representation marker. The generator explicitly avoids inventing Python or Bash replacements before evaluation evidence exists. Deterministic infrastructure, such as approved datasource tools, may exist from the outset; the English-only claim concerns the SOP representation, not every component of the runtime system.

`pland-evolver` receives the generated agent, labeled development and validation evaluations, a configured runner, a task-local scorer, a quality policy, an optimization metric, and resource limits. Every development case is run from isolated state. The system stores output, trace, latency, tokens, estimated cost, errors, and an immutable snapshot of the SOP.

The evolver is itself an Agent Skill. It instructs the host reasoning/coding agent to cluster development failures and unnecessary expense and propose one bounded change. For

every SOP step, exactly one representation is retained:

1. **English instruction:** model-mediated semantic judgment.
2. **Reference:** one-level supporting procedure; a reference is not automatically deterministic.
3. **Command:** explicit Python or Bash that performs, controls, validates, or replaces the corresponding step.

A command counts as deterministic only when the evaluated runtime invokes it and consumes its result. A script that merely produces text for another model decision is not treated as deterministic substitution. PLaND prohibits hiding an LLM call inside a command classified as deterministic.

The current evolver policy requires a command candidate to preserve the complete previously accepted English path as fallback and prohibits shortening that path in the same bounded change. Preconditions and guards can be derived from requirements, policy, tool schemas, and development traces. On precondition, execution, verification, or guard failure, execution must escape to the English path.

The implementation therefore contains two distinct forms of automation. Candidate proposal is agentic: a capable host agent is instructed to use development evidence to form a bounded hypothesis and, where appropriate, create code. Candidate promotion is deterministic: framework-level and task-local comparison scripts verify frozen invariants and evaluate quality and expense.

The repository does not contain a standalone general-purpose pattern-mining or rule-induction algorithm. It does not specify a universal clustering algorithm, rule language, support threshold, or statistical induction procedure. This matters for claim scope: PLaND specifies a trace-guided host-agent evolution policy and a rigorous promotion mechanism, but the present experiments do not prove general autonomous rule discovery.

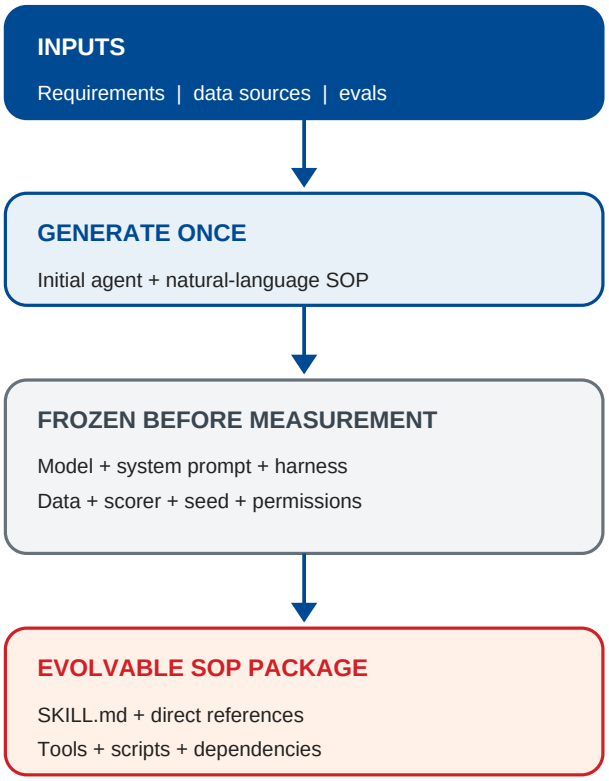

Figure 2. Frozen PLaND boundary. The initial agent and comparison contract are finalized before baseline measurement. Candidate changes are restricted to the workflow skill package.

Frozen boundary, fallback, and acceptance

Before baseline measurement, the generic PLaND methodology freezes the model and digest, system prompt, normalized agent harness, evaluation inputs and expected outputs, task-local scorer, datasource snapshot, seed, runtime settings, execution permissions, quality target, baseline policy, and expense objective. Only the workflow skill package may evolve: `SKILL.md`, directly referenced instructions, directly invoked Python/Bash scripts or tools, and approved dependencies.

Baseline viability is checked before expense optimization. A nonviable baseline stops the experiment. Repairing the model, harness, perception stack, tool interface, evaluator, or execution budget requires a new frozen experiment.

The generic framework-level assessor determines whether a candidate is eligible for validation or must be rejected. Experiment-specific comparators can impose stronger guardrails. In the text-classification confirmatory studies, the comparator calculates Wilson intervals, paired bootstrap intervals, exact McNemar tests, token-reduction intervals, and route-level metrics. The later quality-first study additionally requires no observed accuracy regression, a tighter paired lower bound, per-label recall protection, minimum command-route precision, and minimum token reduction.

Conceptually, PLaND seeks a candidate W' that reduces E(W) subject to the quality contract:

$E(W') < E(W)$,

$Q(W') >= Q_{min}$,

and, when configured,

Q(W') - Q(W) >= -epsilon.

This is a bounded search, not a proof of global optimality. Candidate attempts are capped at ten by default and may stop earlier because of cost, time, or no-improvement limits.

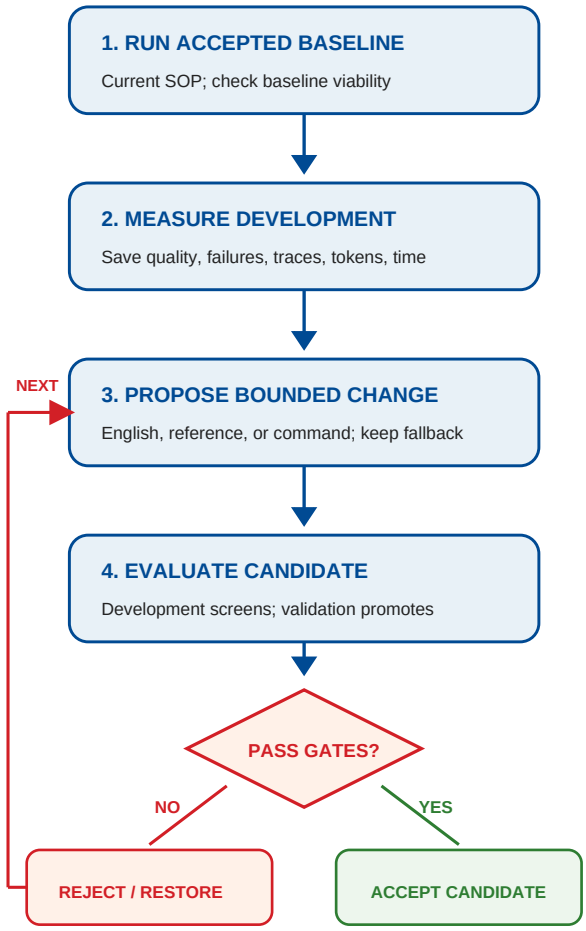

Figure 3. One bounded PLaND evolution cycle. Development evidence can motivate a candidate; validation promotes it only when the frozen quality and expense gates pass.

Benchmark execution semantics

The principal text benchmarks use a shared frozen harness. For hybrid runs, the harness imports the configured deterministic classifier and calls it first. If the classifier returns one valid label, the case is recorded as command-routed and no model call is made. If it abstains, the harness constructs the model prompt from the frozen system prompt, SOP, allowed labels, and case text and invokes the local model.

Thus the confirmatory text experiments test the **computational routing semantics** of a hybrid SOP: deterministic route versus model fallback. They do not test autonomous DeepAgent interpretation of the instruction "run classify.py" during each benchmark case. End-to-end DeepAgent command interpretation remains a separate system-integration question.

The text runner hashes the system prompt, evaluation file, selection manifest, classifier, scorer/harness, model digest, seed, and runtime contract. Dataset snapshots and provenance artifacts separately record source revision, selection rules, and source identifiers. The generic PLaND comparator additionally supports explicit datasource-snapshot hashes in the full framework contract.

Experimental evidence hierarchy

The repository contains multiple generations of experiments. To avoid mixing different protocols, this paper separates them into four evidence classes.

Exploratory pilots and robustness studies are small earlier experiments used to test mechanisms, candidate boundaries, OCR conditions, or failure modes. Some predate the current frozen mutation policy.

Confirmatory studies use larger prepared splits and a frozen protocol. For the original classification protocol, both variants must reach 0.80 accuracy; the paired 95% bootstrap lower bound for hybrid minus natural-language accuracy must be at least -0.02; token reduction must be at least 5% with a positive paired lower bound. Receipt extraction instead requires field F1 of at least 0.50. Text datasets use 100 development, 100 validation, and 1,000 test cases. Test is released only after validation passes.

Optimized replications rerun frozen already-selected skills to characterize reproducibility. They do not constitute new candidate discovery. When a test set was already opened, they are replications rather than new untouched test evaluations.

Quality-first fresh validations use new 500-case validation sets that exclude all 1,200 cases from each earlier confirmatory text selection. They preserve the complete NL fallback and apply a stricter current-policy-aligned release contract. Three paired seeds are run, and the test splits remain closed unless every validation gate passes.

Data and runtime

Public datasets include LEDGAR legal clauses [12], CFPB complaint narratives [13], SpamAssassin email [14], SROIE receipts [15], and RVL-CDIP/Tobacco3482 document images [16, 17]. LiteParse supplied an OCR condition rather than a separate dataset [18]. Dataset preparation records source identifiers, hashes, split membership, exclusions, and pilot-overlap checks.

The original confirmatory text runs used local Ollama and `qwen3:14b`. The principal optimized replications used Ollama 0.33.0 with `qwen3:14b`, digest `bdbd181c33f2ed1b31c972991882db3cf4d192569092138a7d29e973cd9debe8`, on an Apple M5 Pro with 48 GB unified memory. Thinking and streaming were disabled; temperature was 0; context was 4,096 tokens; output cap was 128 tokens; an exact JSON schema was used; the model was preloaded and retained; Flash Attention and q8_0 KV cache were enabled; and two parallel requests were used.

The optimized variance study used seeds 20260903, 20260904, and 20260905. The quality-first fresh validation used seeds 20260906, 20260907, and 20260908, reversing NL/candidate order for the middle seed. Both studies use the same model digest and two-worker runtime.

Original confirmatory evidence

The original confirmatory studies provide the primary held-out evidence because their release policy was frozen before the expanded tests were opened.

Results

Table 1. Original confirmatory results

Workflow and split	Quality (NL -> hybrid)	Tokens (NL -> hybrid)	Decision
LEDGAR validation (100)	92.0% -> 93.0% accuracy	37,147 -> 21,104 (-43.19%)	Pass; test released
LEDGAR test (1,000)	93.5% -> 92.7% accuracy	376,088 -> 225,573 (-40.02%)	Pass
CFPB validation (100)	79.0% -> 72.0% accuracy	60,514 -> 35,247 (-41.75%)	Reject; test closed
SpamAssassin validation (100)	90.0% -> 86.0% accuracy	238,077 -> 228,359 (-4.08%)	Reject; test closed
QS-OCR/Tobacco validation (100)	70.0% -> not run	1,082,556 -> not run	Baseline nonviable
SROIE LiteParse validation (100)	0.344 field F1 -> not run	49,768 -> not run	Baseline nonviable
RVL mirror validation (100)	50.0% -> not run	1,072,199 -> not run	Baseline nonviable

LEDGAR validation passed the frozen release rule, so its 1,000-case test was opened once. On that untouched test, hybrid minus NL accuracy was -0.8 percentage points with a paired 95% interval of [-1.6, 0.0], within the two-point margin. Model calls fell from 1,000 to 589. The 411 command-routed cases contributed 146,366 of 150,515 saved tokens, or 97.24%; only 2.76% of the token savings came from the shorter fallback prompt used in that earlier hybrid.

This caveat is important. The original LEDGAR hybrid preserved the semantic fallback behavior but also shortened the NL fallback instructions when introducing the command. The **current** evolver policy no longer permits those two changes in one candidate: it requires the complete accepted English path to remain unchanged, with any later prompt compression evaluated separately. The earlier LEDGAR result is therefore retained as valid evidence under its frozen protocol, but it is not treated as a perfect implementation of the later stricter mutation policy.

CFPB lost seven accuracy points and its NL baseline also missed the 80% floor, so test remained closed. SpamAssassin lost four points and saved only 4.08% tokens, so it also failed the release contract. The three multimodal confirmatory tracks stopped after nonviable baselines and did not proceed to hybrid testing.

Optimized replications of frozen text skills

Three later paired runs used seeds 20260903-20260905 and reran the frozen text-classification variants **without regeneration or evolution**.

Table 2. Three-run optimized replication results

Workflow and split	Accuracy mean (NL -> hybrid)	Mean tokens (NL -> hybrid)	Routing / decision
LEDGAR test (1,000)	93.7% (SD 0) -> 92.9% (SD 0)	376,090 -> 225,575.33 (-40.02%)	411 command / 589 model; pass in 3/3
CFPB validation (100)	78.0% (SD 0) -> 71.0% (SD 0)	58,372 -> 33,120 (-43.26%)	42 command / 58 model; reject in 3/3
SpamAssassin validation (100)	88.0% (SD 0) -> 85.0% (SD 0)	188,384 -> 178,880.67 (-5.04%)	3 command / 97 model; reject in 3/3

Each LEDGAR replication reproduced the -0.8-point NL/hybrid difference and approximately 40.02% token reduction. Every variant produced identical labels across the three seeds within its own condition, so observed cross-run disagreement was zero for both NL and hybrid. Routing was also identical. Because both variants reached the same zero-disagreement floor under the temperature-zero runtime, this study does not demonstrate hybrid-specific variance reduction.

These repeated LEDGAR test runs are reproducibility evidence on an already-opened test set, not new confirmatory releases.

Fresh quality-first validation under the current stricter policy

A subsequent study applied one dataset-independent quality-first policy to fresh 500-case validation sets for LEDGAR, CFPB, and SpamAssassin. Each new selection excluded all 1,200 cases from that dataset's prior confirmatory selection. The complete NL fallback was preserved in the quality-first SOP. The quality-first release contract required:

1. absolute accuracy of at least 0.80;
2. no observed accuracy regression;
3. a paired accuracy lower bound of at least -0.5 percentage points;
4. maximum per-label recall drop of 2 percentage points;
5. command-route precision of at least 99%;
6. token reduction of at least 5% with a positive paired lower bound.

Three paired seeds, 20260906-20260908, produced identical predictions and token counts within each dataset/variant.

Table 3. Fresh quality-first validation results

Dataset	Accuracy (NL -> quality-first)	Difference (95% interval)	Efficiency / route / outcome
LEDGAR	93.6% -> 92.8%	-0.8 pp [-1.6, -0.2]	8.36% token reduction; 82/500 at 98.78% precision; reject
CFPB	74.8% -> 73.8%	-1.0 pp [-2.4, +0.2]	-2.25% token reduction; 12/500 at 100% precision; reject
SpamAssassin	90.2% -> 89.0%	-1.2 pp [-2.2, -0.4]	-0.59% token reduction; 3/500 at 100% precision; reject

Negative token reduction means the candidate used more tokens than NL. All three candidates failed the frozen quality-first release policy in all three replications, and no test split was opened.

LEDGAR remains informative. A more conservative command layer routed only 82 of 500 cases and saved 8.36% tokens, but command precision was 98.78%, below the 99% route guardrail, and the candidate also failed the no-regression, paired-lower-bound, and per-label requirements. This fresh study therefore does not overturn the earlier confirmatory result; it answers a different question under a stricter current-policy contract and shows that the acceptable deterministic frontier depends on the quality policy.

Earlier pilot and robustness evidence

The repository also contains smaller studies that predate or sit outside the confirmatory protocol.

Table 4. Earlier pilot and robustness studies

Study	Key result	Evidence boundary
QS-OCR/Tobacco original pilot	Constrained agent preserved 100% validation accuracy and reduced validation tokens 46.40%, but the accepted SOP had no command step.	Pre-protocol agent-boundary optimization; not English-to-command SOP compilation.
QS-OCR/Tobacco schema-v2	Genuine hybrid preserved 100% validation accuracy on 10 cases, reduced tokens 23.30%, and reduced mean latency 23.16%.	Accepted operation-level proof-of-concept; one case per class, one seed, no third held-out split.
SpamAssassin pilot	Across 20 cases, accuracy was 90% NL vs 85% hybrid; tokens fell 15.27% and calls 20 -> 16. Four-case held-out test was 100% vs 75%.	Accepted under an earlier 75% absolute threshold; threshold-level pilot evidence only.
RVL-CDIP schema-v2	Validation accuracy improved 81.25% -> 87.50%, tokens fell 34.36%, and mean latency fell 29.17%.	Rejected because 87.50% remained below the frozen 90% quality floor.
SROIE pilot / OCR replications	Frozen-OCR hybrid reduced tokens 7.28% overall but validation field F1 fell 0.746 -> 0.715; end-to-end OCR baselines were below viability.	Negative evidence; OCR quality dominated and no accepted hybrid was established.

These pilots clarify mechanism and failure modes but are not pooled with the later confirmatory or quality-first estimates.

Claim-to-evidence boundary

Table 5. What the current repository establishes

Claim	Evidence status
A workflow SOP can combine model-mediated and deterministic representations	Implemented and exercised
A command can resolve a bounded region and abstain to model fallback	Implemented and exercised in text routing
Deterministic routing can bypass model calls and materially reduce tokens	Demonstrated in the original LEDGAR confirmatory test
Token savings alone are insufficient for acceptance	Demonstrated repeatedly by CFPB, SpamAssassin, RVL, SROIE, and quality-first rejections
Current-policy candidates preserve the complete accepted NL fallback	Implemented in the evolver policy and quality-first SOPs
The evolver skill can instruct a host agent to use development traces to propose bounded code changes	Implemented as Agent Skill policy
General autonomous pattern discovery from arbitrary histories	Not independently implemented or benchmarked
Researcher-independent trace-to-rule synthesis produced the reported LEDGAR rule set	Not established
Autonomous DeepAgent execution of command steps produced the main text benchmark results	Not tested; routing was implemented by the benchmark harness
Continuous production self-modification	Not implemented; monitoring/demotion remain prospective
Global least-nondeterministic optimum	Not established; search is bounded
Dollar, energy, or carbon savings	Not measured; tokens/calls are expense proxies

Discussion

Business interpretation: model reasoning as a resource, not a default

The main systems argument is that non-deterministic reasoning should be allocated where it is needed rather than applied uniformly across a workflow. Business processes often contain both stable and ambiguous decisions. When stable regions are repeatedly routed through a language model, the workflow spends model calls and tokens on work that ordinary computation may be able to execute directly.

PLaND provides a controlled mechanism for moving that boundary. The English baseline acts as an expressive default when the structure of the task is not fully known. Development traces expose errors, repeated reasoning, and expense. The evolver skill can instruct a host reasoning agent to formulate a bounded candidate. Deterministic evaluation then decides whether that candidate is promotable. This structure is particularly relevant to domains such as fraud detection, where known patterns may be encoded directly while novel narratives or combinations continue to require contextual reasoning.

The important unit is not necessarily an entire workflow step. LEDGAR shows that a semantic decision itself can contain a deterministic region. Contract-clause classification remains a semantic problem overall, but selected unambiguous patterns can be resolved by explicit rules while the residual region falls back to the model. This is a stronger interpretation than merely moving obviously mechanical operations, such as counting or schema validation, into code.

The deterministic frontier depends on the quality contract

The original LEDGAR confirmatory experiment and the later quality-first validation should be interpreted together rather than as contradictory studies.

Under the earlier frozen protocol, a broader deterministic route replaced 411 of 1,000 model calls and reduced tokens 40.02% while remaining within a prespecified two-point non-inferiority margin. That result passed its protocol.

The later quality-first study preserved the complete NL fallback, used fresh validation data, narrowed deterministic coverage, and imposed substantially stricter requirements: no observed accuracy regression, a 0.5-point paired lower bound, per-label recall protection, 99% command precision, and at least 5% token reduction. Under that contract, LEDGAR's 8.36% token saving was not sufficient and the candidate was rejected.

This is consistent with the core PLaND thesis. "Least non-determinism" is not an unconditional drive toward maximum deterministic coverage. It is the minimum model-mediated execution that is justified by the configured quality contract and available evidence. A stricter contract can move the acceptable frontier toward more model reasoning.

Progressive compilation and workflow maturity

PLaND's intended evolution can be viewed as progressive compilation. Early in a workflow's life, broad model mediation can be rational because the decision surface is not fully understood. As evidence accumulates, stable behavior can become a candidate for a more deterministic representation. The result may be lower model use without requiring the workflow to become fully deterministic.

However, the present implementation is intentionally bounded. Development evidence can motivate a candidate within an experiment, but validation and held-out evidence are

protected from candidate mining. Once held-out results are opened, evolution stops. Continuous production learning would therefore require a sequence of separately frozen experiments rather than silent self-modification.

This distinction is especially important for drifting domains. A fraud rule that was stable at time t may cease to be reliable at time t+1. A mature longitudinal PLaND system should therefore support both promotion of newly stable regions and demotion of rules that no longer satisfy the quality contract. The current repository describes canary monitoring and restoration only prospectively.

What the code automates today

The code supports more than passive validation of a manually supplied rule, but less than a complete automatic rule-learning system. `pland-evolver` is an Agent Skill. It tells a host reasoning/coding agent to inspect development traces, cluster failures and unnecessary expense, and propose one bounded change. It permits that host agent to create Python or Bash candidates and to derive preconditions, guards, and fallback behavior from development evidence.

The deterministic Python scripts then enforce experimental discipline. Framework-level assessors verify identity and hashes, compare frozen invariants, evaluate quality and expense, and decide whether a candidate can proceed. Task-local comparators can impose stronger statistical and route-specific guardrails.

What is absent is a separately defined pattern-discovery algorithm with an independently evaluated trace-to-rule benchmark. The large text replications rerun frozen skills rather than showing researcher-independent candidate synthesis. It would therefore be inaccurate to claim that the present experiments prove general autonomous pattern discovery or code generation from arbitrary execution histories.

This boundary identifies the next testable research question: whether a host agent can reliably perform the trace-to-candidate step under a frozen protocol, with no researcher-authored rule content, and produce code that survives PLaND's acceptance gates.

Cost interpretation

The original accepted LEDGAR hybrid reduced model calls from 1,000 to 589 and tokens from 376,088 to 225,573. The mechanism decomposition shows that 97.24% of saved tokens came from bypassing model calls and 2.76% from fallback prompt shortening. This demonstrates the mechanism through which deterministic compilation can reduce inference expense.

The present local Ollama experiments record no paid per-call service charge, so the paper does not convert these savings into dollars. Actual production savings would depend on provider pricing, caching, context length, input/output mix, hardware utilization, and the execution cost and maintenance burden of deterministic code itself.

PLaND's expense abstraction is broader than tokens. The evolver can optimize total tokens, mean latency, or estimated model cost and instructs candidate generation to consider CPU, memory, storage, network, service charges, caching, parallelism, and repeated setup work. The reported evidence is strongest for model calls and tokens.

Negative results are central

The new quality-first study reinforces an important point already visible in CFPB, SpamAssassin, RVL-CDIP, and SROIE: determinism cannot be the objective by itself. Several candidates reduced tokens or latency yet failed a quality floor, non-inferiority rule, per-label guardrail, or route-precision requirement. PLaND rejected them.

A useful interpretation is therefore not "PLaND makes workflows deterministic." It is "PLaND tests how much model-mediated work can be removed under a specified quality contract." The accepted endpoint can remain highly model-mediated.

Limitations

The central accepted held-out result is a balanced top-10 single-label LEDGAR subset, not the full multi-label distribution or a production legal workload. The deterministic LEDGAR rules are domain-specific and their transferability is unknown.

The original accepted LEDGAR hybrid does not exactly satisfy the current mutation policy because it shortened the semantic fallback when introducing the command. The repository's own mechanism decomposition isolates this effect and attributes 2.76% of token savings to prompt shortening. The later quality-first study corrects the protocol issue by preserving the complete fallback, but its candidates are rejected under the stricter contract.

The principal text benchmarks use harness-level routing around frozen SOPs rather than autonomous DeepAgent interpretation and execution of the command step. The generalized generator/evolver policy is therefore evaluated more strongly at the representation, routing, and acceptance level than as a fully autonomous end-to-end self-evolving DeepAgent.

The candidate-generation policy is agentic, but autonomous general rule induction is not independently measured. The successful schema-v2 document pilot is small and has no third held-out split or repeated seeds. The quality-first study uses fresh validation data but does not open test because all candidates fail.

The current benchmarks are classification and extraction tasks resembling individual decision nodes, not long-running business processes with persistent state, multiple transactional tools, human intervention, or irreversible actions. The business-process model is therefore a motivating systems abstraction; the empirical evidence is presently at the operation or decision-region level.

Quality thresholds are engineering choices rather than universal standards. The original confirmatory protocol uses a two-point non-inferiority margin, whereas the later quality-first protocol is much stricter. One local model, one machine, temperature-zero decoding, and small numbers of seeds limit generalization. Zero observed cross-run disagreement does not establish general stochastic reliability.

Finally, the study measures tokens, calls, quality, and selected latency metrics. It does not directly measure dollar cost, energy, carbon, maintenance burden, or long-term rule drift.

Future work

The most important next experiment is a trace-to-rule synthesis study. A workflow should begin with an English-only SOP. The host evolver agent should receive only task requirements, development traces, and scorer feedback; researcher-authored deterministic rule content should be prohibited. The agent should be required to produce the candidate hypothesis, rule or program, preconditions, guards, and fallback. The complete generation transcript should be preserved. Candidate code should then be evaluated through the existing frozen PLaND gates on untouched validation and held-out data. Repeating this process across tasks and models would directly test whether PLaND can autonomously discover and compile stable decision regions.

A second priority is a stateful business-process benchmark containing heterogeneous nodes: interpretation, extraction, deterministic calculation, policy application, external tool use, exception handling, and final-state transitions. Evaluation should score final state, permissible actions, tool correctness, policy compliance, fallback behavior, process latency, and expense rather than classification accuracy alone.

A third direction is longitudinal evolution. Production evidence collected during a defined period could seed a new frozen evolution cycle. Newly stable regions could be promoted, while existing deterministic rules that fail canary or drift criteria could be restored to the previously validated English path. This would test the broader vision of PLaND as a controlled self-compiling workflow architecture without allowing unvalidated online self-modification.

Further work should compare models, nonzero sampling temperatures, alternative candidate-generation agents, explicit rule languages, program synthesis methods, route precision and recall, tool-sequence variability, monetary inference cost, and maintenance overhead.

Conclusion

Business processes and agentic systems do not require a single computational representation. Some decisions require model judgment; others can be executed deterministically; and some semantic decisions can contain stable deterministic regions alongside an ambiguous residual.

PLaND provides an evaluation-driven path for moving that boundary. It begins with an English SOP, records development evidence, lets a host reasoning agent propose bounded representation changes, and promotes a deterministic substitution only when frozen task-local quality and expense gates pass. Current-policy command candidates preserve the complete accepted English fallback.

The strongest positive held-out result is the original LEDGAR confirmatory test: 411 of 1,000 cases bypassed the model, tokens fell 40.02%, and the -0.8-point accuracy difference remained within the prespecified two-point margin. Three optimized replications reproduced the same qualitative outcome. A later quality-first study used fresh 500-case validation sets and stricter current-policy guardrails; LEDGAR, CFPB, and SpamAssassin candidates were all rejected and no tests were opened. The combination of accepted and rejected results is the central empirical message.

The code is designed for trace-guided candidate evolution, but the completed experiments do not prove general autonomous pattern discovery or researcher-independent rule induction. They demonstrate the downstream mechanism such a system requires: selective deterministic execution, abstention and fallback, frozen comparison, explicit rejection, and measurable reduction of model-mediated work when the configured evidence permits it.

The practical principle is therefore: start with model judgment when the structure is unknown; codify stable behavior when evidence supports it; keep genuinely ambiguous behavior model-mediated; and never trade away the quality contract merely to reduce model use.

Acknowledgements

None.

Disclosure and Conflict of Interest

This work received no external funding. The authors declare no conflicts of interest.

Data and Code Availability

Source code, methodology skills, preparation scripts, frozen protocols, result summaries, safe traces, comparisons, and SHA-256 manifests are available in the public PLaND repository at https://github.com/mnvsk97/PLaND. This manuscript is aligned to repository commit `f9aa70753c64bd6d409b9ff6ae934edd1f7bedc4`. The repository distinguishes exploratory pilots, original confirmatory studies, optimized replications, and fresh quality-first validations. The three-run text variance study reruns frozen skills and is reported as replication evidence rather than new candidate evolution. The quality-first study uses fresh validation sets and keeps all corresponding test splits closed because every candidate failed at least one release gate. Large copyrighted or sensitive source records are not redistributed; source references, selection procedures, and hashes are retained for authorized reproduction.

References

[1] Agent Skills. (2026). *Agent Skills specification*. https://agentskills.io/specification

[2] LangChain. (2026). *DeepAgents: Skills*. https://docs.langchain.com/oss/python/deepagents/skills

[3] LangChain. (2026). *LangGraph overview*. https://docs.langchain.com/oss/python/langgraph/overview

[4] Khattab, O., et al. (2023). DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv*. https://arxiv.org/abs/2310.03714

[5] Li, Z., et al. (2024). AutoFlow: Automated workflow generation for large language model agents. *arXiv*. https://arxiv.org/abs/2407.12821

[6] Zhang, J., et al. (2024). AFlow: Automating agentic workflow generation. *arXiv*. https://arxiv.org/abs/2410.10762

[7] Hu, S., Lu, C., & Clune, J. (2024). Automated design of agentic systems. *NeurIPS*. https://arxiv.org/abs/2408.08435

[8] Yang, Y., et al. (2026). SkillOpt: Executive strategy for self-evolving agent skills. *arXiv*. https://arxiv.org/abs/2605.23904

[9] Liu, Y., et al. (2026). SkillRevise: Improving LLM-authored agent skills via trace-conditioned skill revision. *arXiv*. https://arxiv.org/abs/2606.01139

[10] Gao, Y., et al. (2026). SkillReducer: Optimizing LLM agent skills for token efficiency. *arXiv*. https://arxiv.org/abs/2603.29919

[11] Kevin, C., et al. (2026). Evaluating skills, not just agents: Agentic continuous evaluation of skills. *arXiv*. https://arxiv.org/abs/2608.20614

[12] Tuggener, D., et al. (2020). LEDGAR: A large-scale multi-label corpus for text classification of legal provisions in contracts. *LREC 2020*, 1235-1241. https://aclanthology.org/2020.lrec-1.155/

[13] Consumer Financial Protection Bureau. (2026). *Consumer Complaint Database*. https://www.consumerfinance.gov/data-research/consumer-complaints/

[14] Apache SpamAssassin. (2006). *Public corpus*. https://spamassassin.apache.org/old/publiccorpus/

[15] Huang, Z., et al. (2021). ICDAR2019 competition on scanned receipt OCR and information extraction. *arXiv*. https://arxiv.org/abs/2103.10213

[16] Harley, A. W., Ufkes, A., & Derpanis, K. G. (2015). Evaluation of deep convolutional nets for document image classification and retrieval. *ICDAR 2015*. https://arxiv.org/abs/1502.07058

[17] Lim, G., Larson, S., & Leach, K. (2024). Label errors in the Tobacco3482 dataset. *arXiv*. https://arxiv.org/abs/2412.13140

[18] LlamaIndex. (2026). *LiteParse: Open-source document parsing*. https://github.com/run-llama/liteparse